

COMPILADORES E INTÉRPRETES

Proyecto
Reglas de Sintaxis de MINIJAVA
Segundo Cuatrimestre de 2026

1 Introducción

En este documento se describen las reglas de sintaxis de MINIJAVA. Como se menciona en el documento de especificación del proyecto, MINIJAVA es un lenguaje que puede verse como una simplificación de Java, donde por una parte no se consideran elementos avanzados como Excepciones o Hilos, y por otra parte la estructura de su sintaxis es más estricta. En este documento se presentará la estructura sintáctica del lenguaje mediante su gramática. A partir de las producciones de esta gramática se podrán identificar con mayor facilidad aquellos elementos que alejan a MINIJAVA de Java.

MINIJAVA incorpora una versión restringida de genericidad, interfaces y arreglos. Las restricciones correspondientes a estas construcciones pueden identificarse a partir de las producciones de la gramática presentada en este documento.

2 Gramática de MiniJava

Cualquier programa MINIJAVA sintácticamente válido debe ser producto de la gramática que se presenta en esta sección. La gramática sigue la notación BNF-extendida, donde:

terminal	es un símbolo terminal
<Clase>	es un símbolo no terminal (además la primer letra es una mayúscula)
ϵ	representa la cadena vacía
<X>::=α	representa una producción, con α una secuencia de terminales y no terminales
<X>::=$\alpha \beta$	es una abreviación de <X>::=α y <X>::=β

Producciones BNF

El no terminal de inicio es <Inicial> y las producciones de la Gramática de MINIJAVA son:

```
<Inicial> ::= <ListaClases> eof

<ListaClases> ::= <Clase> <ListaClases> | <Interfaz> <ListaClases> |  $\epsilon$ 

<Clase> ::= class idClase <GenericidadOpcional> <HerenciaOpcional> { <ListaMiembros> }

<Interfaz> ::= interface idClase <GenericidadOpcional> <ExtensionOpcional> { <ListaMetodosInterfaz> }

<GenericidadOpcional> ::= < idGen > |  $\epsilon$ 

<HerenciaOpcional> ::= extends <TipoReferencia> | implements <TipoReferencia> |  $\epsilon$ 

<ExtensionOpcional> ::= extends <TipoReferencia> |  $\epsilon$ 
```

```

<ListaMiembros> ::= <Miembro> <ListaMiembros> | ε

<ListaMetodosInterfaz> ::= <MetodoInterfaz> <ListaMetodosInterfaz> | ε

<Miembro> ::= <Atributo> | <Metodo> | <Constructor>

<Atributo> ::= <Tipo> idMetVar ;

<Metodo> ::= <ModificadorOpcional> <TipoMetodo> idMetVar <ArgsFormales> <Bloque>

<MetodoInterfaz> ::= <TipoMetodo> idMetVar <ArgsFormales> ;

<Constructor> ::= public idClase <ArgsFormales> <Bloque>

<ModificadorOpcional> ::= static | ε

<TipoMetodo> ::= <Tipo> | void

<Tipo> ::= <TipoBase> <DimensionesOpcionales>

<TipoBase> ::= <TipoPrimitivo> | <TipoReferencia> | idGen

<DimensionesOpcionales> ::= []<DimensionesOpcionales> | ε

<TipoReferencia> ::= idClase <TipoGenericoOpcional>

<TipoPrimitivo> ::= boolean | char | int

<TipoGenericoOpcional> ::= < <InstanciadoOParametrico> > | ε

<InstanciadoOParametrico> ::= idGen | idClase

<ArgsFormales> ::= ( <ListaArgsFormalesOpcional> )

<ListaArgsFormalesOpcional> ::= <ListaArgsFormales> | ε

<ListaArgsFormales> ::= <ArgFormal>
<ListaArgsFormales> ::= <ListaArgsFormales> , <ArgFormal>

<ArgFormal> ::= <Tipo> idMetVar

<Bloque> ::= { <ListaSentencias> }

<ListaSentencias> ::= <Sentencia> <ListaSentencias> | ε

<Sentencia> ::= ;
<Sentencia> ::= <Asignacion> ;
<Sentencia> ::= <Llamada> ;
<Sentencia> ::= <VarLocal> ;
<Sentencia> ::= <Return> ;
<Sentencia> ::= <If>
<Sentencia> ::= <While>
<Sentencia> ::= <Bloque>

<Asignacion> ::= <Expresion>

<Llamada> ::= <Expresion>

```

```

<VarLocal> ::= var idMetVar = <ExpresionCompuesta>

<Return> ::= return <ExpresionOpcional>

<ExpresionOpcional> ::= <Expresion> |  $\epsilon$ 

<If> ::= if ( <Expresion> ) <Sentencia>
<If> ::= if ( <Expresion> ) <Sentencia> else <Sentencia>

<While> ::= while ( <Expresion> ) <Sentencia>

<Expresion> ::= <ExpresionCompuesta> <OperadorAsignacion> <ExpresionCompuesta>
<Expresion> ::= <ExpresionCompuesta>

<OperadorAsignacion> ::= =

<ExpresionCompuesta> ::= <ExpresionCompuesta> <OperadorBinario> <ExpresionBasica>
<ExpresionCompuesta> ::= <ExpresionBasica>

<OperadorBinario> ::= || | && | == | != | < | > | <= | >= | + | - | * | / | %

<ExpresionBasica> ::= <OperadorUnario> <Operando>
<ExpresionBasica> ::= <Operando>

<OperadorUnario> ::= + | - | !

<Operando> ::= <Primitivo>
<Operando> ::= <Referencia>

<Primitivo> ::= true | false | intLiteral | charLiteral | null

<Referencia> ::= <Primario>
<Referencia> ::= <Referencia> <VarEncadenada>
<Referencia> ::= <Referencia> <MetodoEncadenado>
<Referencia> ::= <Referencia> <AccesoArreglo>

<Primario> ::= this
<Primario> ::= stringLiteral
<Primario> ::= <AccesoVar>
<Primario> ::= <CrearArreglo>
<Primario> ::= <LlamadaConstructor>
<Primario> ::= <LlamadaMetodo>
<Primario> ::= <LlamadaMetodoEstatico>
<Primario> ::= <ExpresionParentizada>

<AccesoVar> ::= idMetVar

<CrearArreglo> ::= new <TipoBase> <DimensionesConTamano>

<LlamadaConstructor> ::= new <TipoReferencia> <ArgsActuales>

<ExpresionParentizada> ::= ( <Expresion> )

<LlamadaMetodo> ::= idMetVar <ArgsActuales>

<LlamadaMetodoEstatico> ::= idClase . idMetVar <ArgsActuales>

```

$\langle \text{DimensionesConTamano} \rangle ::= [\langle \text{Expresion} \rangle] \langle \text{DimensionesConTamano} \rangle \mid [\langle \text{Expresion} \rangle]$

$\langle \text{ArgsActuales} \rangle ::= (\langle \text{ListaExpsOpcional} \rangle)$

$\langle \text{ListaExpsOpcional} \rangle ::= \langle \text{ListaExps} \rangle \mid \epsilon$

$\langle \text{ListaExps} \rangle ::= \langle \text{Expresion} \rangle$

$\langle \text{ListaExps} \rangle ::= \langle \text{Expresion} \rangle , \langle \text{ListaExps} \rangle$

$\langle \text{VarEncadenada} \rangle ::= . \text{idMetVar}$

$\langle \text{MetodoEncadenado} \rangle ::= . \text{idMetVar} \langle \text{ArgsActuales} \rangle$

$\langle \text{AccesoArreglo} \rangle ::= [\langle \text{Expresion} \rangle]$